

Offer-to-SKU Matching

Problem Statement and Modelling Brief

Document ID	CI-2026-014
Version	1.2 (supersedes 1.1, which used the pre-August labelling guide)
Date	20 September 2026
Owner	Catalog Intelligence — Entity Resolution pod
Stakeholders	Catalog Operations, Pricing, Search Relevance, Merchant Trust
Status	Approved for build. Milestone 1 kickoff.
Audience	Data scientist / ML engineer assigned to the matcher rebuild

Contents

1. Executive summary	2
2. Business context	2
3. Why the current system is failing	3
4. Problem statement	4
5. Identity specification (the business rules)	4
6. Scope and non-goals	6
7. The data	6
8. Labelling process and known label noise	9
9. Evaluation protocol and success criteria	10
10. Incumbent baseline (numbers to beat)	11
11. Catalogue of known hard cases	12
12. Engineering constraints	13
13. Deliverables and milestones	14
Appendix A — File manifest	15
Appendix B — Glossary	15

1. Executive summary

Meridian Commerce Group operates a shopping comparison surface that ingests product offers from five marketplace feeds. Before an offer can appear on a product page, it has to be attached to the correct internal SKU. That attachment is currently made by a hand-maintained rule set (catalog-matcher v2.3), and its precision has degraded to the point where Catalog Operations no longer trusts it to run unsupervised.

On our September evaluation set the incumbent rules achieve **precision 0.400 and recall 0.658**. In plain terms: three out of every five offers the system merges into an existing SKU are merged into the *wrong* SKU, and it misses a third of the genuine duplicates it is shown. Every false merge is customer-visible — a 512 GB phone listed at the 256 GB price, a replacement filter sitting on the espresso machine's buy box — and each one is currently caught, if at all, by a human reviewer or a customer complaint.

This brief defines the matching problem precisely, specifies what "the same product" means for our catalogue, describes the labelled dataset that has been assembled for the rebuild, and fixes the evaluation protocol. The deliverable is a **pairwise classifier that decides whether two offers refer to the same purchasable SKU, operating on the candidate pairs produced by the existing blocking stage.**

The headline requirement: **this is a precision-first problem.** A missed merge costs us a slightly redundant catalogue. A false merge costs us a wrong price on a live product page, a mis-routed order, and a merchant dispute. Optimise accordingly — **and note that the incumbent's headline accuracy of 0.828 looks respectable only because 87% of candidate pairs are negatives.**

2. Business context

Meridian aggregates offers from five sources. Each has its own title conventions, its own attribute completeness, and its own idea of what a product identifier is.

Feed	Character	Title style	Typical problems
ShopAxis	Large managed marketplace	Long, comma-delimited, attribute-rich	Marketing adjectives padded into the title; attribute order varies by seller
MarketNest	Open seller marketplace	Free-form seller-written	Worst quality. Shouting case, emoji, shipping promises, brand often omitted, typos
ValueMart	Big-box retailer feed	Terse, fairly clean	Attributes truncated; colour often dropped entirely
GadgetHub	Specialist electronics retailer	Pipe-delimited, spec-forward	Leads with the model code, which helps; inconsistent delimiters
BrandDirect	Manufacturer storefronts	Canonical	Cleanest source. Treat as the closest thing we have to a reference string

Offers arrive continuously. A newly ingested offer is passed through a blocking stage (brand key, shared rare token, character-shingle LSH) that reduces the full offer $\times$ offer space to a manageable candidate set. The matcher then scores each candidate pair. Pairs scored above threshold are merged into a single SKU node; the SKU's buy box, price history, review pool and search document are all derived from that merged group.

Who depends on the output

- **Pricing** — the buy-box algorithm ranks offers within a SKU group. A foreign offer inside the group can win the buy box at a price that is not achievable for the product shown.
- **Search Relevance** — SKU groups are the retrieval unit. Over-merging collapses distinct results; under-merging splits demand across near-identical pages and depresses both offers' rankings.
- **Merchant Trust** — merchants file disputes when their offer is attached to a product page they did not list against. Disputes are handled manually and have an SLA.
- **Catalog Operations** — currently absorbing the error rate as manual review queue volume.

3. Why the current system is failing

catalog-matcher v2.3 is a three-rule cascade: exact normalised MPN match wins; otherwise a brand mismatch vetoes; otherwise a token-Jaccard score above 0.62 merges. It was written when the catalogue was mostly electronics from three feeds and has been patched incrementally since.

Its failure mode is structural rather than a tuning problem. Jaccard similarity treats every token as equally important, so the token that actually decides identity — 512GB, 48 Pack, XM4 — is outvoted by the fifteen tokens the two titles share. The threshold sweep below shows there is no operating point that rescues it:

Jaccard threshold	Precision	Recall	F1	False merges	Missed merges
0.50	0.298	0.808	0.435	990	100
0.55	0.345	0.746	0.472	737	132
0.62 (current production)	0.400	0.658	0.497	513	178
0.70	0.449	0.579	0.506	369	219
0.80	0.468	0.350	0.400	207	338

Evaluated on the 4,024-pair test split described in §7. Even at a threshold of 0.80, where recall has collapsed to 0.35, precision does not reach 0.47.

The diagnosis: identity in this domain is not a matter of degree. A handful of attributes act as vetoes — capacity, pack quantity, physical size, model generation — and no monotone function of overall string overlap can express "these two strings are 94% identical and therefore definitely not the same product". The rebuild needs a model that can learn conditional, non-additive decision rules over explicitly extracted attributes.

4. Problem statement

Given two offer records a and b drawn from the candidate set produced by blocking, output a score $s(a, b) \in [0, 1]$ estimating the probability that both offers refer to the same purchasable SKU as defined in §5.

Formally the task is binary classification over unordered pairs. Two properties follow from that and are *requirements*, not implementation details:

1. **Symmetry.** $s(a, b)$ must equal $s(b, a)$. Which offer landed in column A is an artefact of how the candidate was written to disk and carries no information. Features derived from a single side (for example the raw length of title A) will let a model learn that artefact. Use symmetric combinations — absolute difference, minimum, maximum, ratio of smaller to larger. Swapping the two columns is a valid augmentation and a valid unit test.
2. **Transitivity is not assumed at this stage.** The model scores pairs. The downstream clustering service is responsible for resolving a set of pairwise decisions into groups, and it applies its own consistency logic. Do not attempt to enforce transitivity inside the pair scorer.

5. Identity specification (the business rules)

This section is the authoritative definition of a match. It was agreed with Catalog Operations in August 2026 and is the same specification given to the annotators. Read it carefully — several of the rules are counter-intuitive and are the source of most disagreement between engineers and the labels.

5.1 Attributes that constitute identity

Two offers are the same SKU if and only if they agree on *all* of the following:

Attribute	Notes	Example of a mismatch
Brand	Manufacturer, not seller or reseller	Sony vs Bose
Product line / model	Including generation	WH-1000XM5 vs WH-1000XM4
Storage or memory capacity	Normalise units before comparing	256 GB vs 512 GB
Physical size / volume / weight	Screen size, pan diameter, cooker litres, tub weight	65" vs 77"
Colour / finish	Marketing colour names count; see §5.3	Aqua vs Whitestone
Pack quantity	Absent means one	16-pack vs 48-pack
Item kind	A product and an accessory for it are never the same SKU	Espresso machine vs its water filter

5.2 Attributes that do **NOT** constitute identity

Condition is an offer attribute, not a SKU attribute. A Renewed, Refurbished, Open Box or Used offer of the Midnight 256 GB iPhone 15 attaches to the *same SKU* as a New one. Condition is carried on the offer and surfaced separately in the buy box. This is the single most common source of engineer confusion, because a condition mismatch is usually accompanied by a large price gap, and price gap is otherwise a useful negative signal. A model that learns "big price difference = different product" will systematically reject genuine refurbished matches.

Also not part of identity: seller, marketplace, price, shipping terms, listing date, promotional bundling language, warranty claims, and country-of-shipment.

5.3 Colour rules

Colour is part of identity, but marketing colour names are not stable across feeds. The same SKU may be "Midnight", "Midnight Black" or simply "Black" depending on the seller; "Natural Titanium" and "Titanium Natural" are the same finish written two ways. Annotators were instructed to judge the underlying finish, not the string. Where a feed omits colour entirely and the rest of the record is unambiguous, the pair was labelled as a match.

5.4 The accessory rule

Accessories are a distinct item kind and never match their host product, even when the host's full name and model code appear in the accessory title — which they almost always do, because that is how accessory sellers get found in search. This is the highest-token-overlap negative class in the dataset. Note that the accessory may share the host's brand (a Dyson replacement filter, a Breville water filter), so a brand-match feature will not save you. Two accessories for different hosts are also a non-match; two offers of the same accessory are a match.

5.5 Worked examples from the labelled data

MATCH (label 1) — same SKU, condition and price differ
A: Breville ClaroSwiss Water Filter [BWF100] fits Breville Barista Express BES870XL 6 pcs | \$7.73 | Refurbished

B: GENUINE ClaroSwiss Water Filter for Breville Barista Express BES870XL 6-Count \$11.49 New Same accessory, same pack size (6), different condition and a 1.5× price gap. Condition is not identity.
MATCH (label 1) – colour written two ways, capacity written two ways A: Apple iPhone 15 Pro Max 256g Titanium Black \$1,192.73 B: Apple iPhone 15 Pro Max - 256GB Black Titanium \$1,132.79 "256g" and "256GB"; "Titanium Black" and "Black Titanium".
NON-MATCH (label 0) – capacity veto A: mu-pelt0s 4 TB Blak \$335.23 mpn=MU PE1T0S B: NEW SEALED Samsung T7 Shield Portable SSD 1.0TB Blk Best Price \$92.20 Identical model code, identical colour, four-fold capacity difference. The MPN match rule in v2.3 merges this pair.
NON-MATCH (label 0) – accessory vs host, shared brand A: Latest Model ClaroSwiss Water Filter for Breville Barista Express BES870XL 6 Pack \$7.94 B: Breville BES870XL Barista Express Espresso Machine Stainless Steel \$660.96 Same brand, same model code in both titles, 83× price ratio.
NON-MATCH (label 0) – generation veto A: NEW SEALED Sony WH1000XM5 Midnight Blue \$366.90 mpn=WH-1000XM5 B: ★ Sony WH-1000XM4 Wireless Noise Canceling Heapdphones SILVER \$280.50 One character of the model code decides it – and it is a character inside an alphanumeric token, invisible to word-level tokenisation.
NON-MATCH (label 0) – size encoded inside the model code A: LG C4 Series OLED evo 4K Smart TV 77-Inch \$3,197.46 mpn=OLED65C4PUA B: LG C4 Series OLED evo 4K Smart TV OLED65 New in Box \$1,948.17 mpn=OLED65C4PUA Screen size appears as "77-Inch" on one side and as the "65" embedded in "OLED65" on the other. Same shared MPN stem.

6. Scope and non-goals

In scope

- Pairwise scoring of candidate offer pairs.
- Feature extraction, normalisation and attribute parsing from the offer record.
- Threshold selection against the operating requirement in §9.
- An error taxonomy and a documented account of which feature addresses which class.

Out of scope for this milestone

- The blocking stage. Candidate pairs are given. Blocking recall is being measured separately by the Retrieval pod (current estimate 0.94).
- Clustering pairwise decisions into groups. Handled downstream.
- Image-based matching. Offer images are available but not in this dataset.
- Catalogue enrichment, attribute imputation, taxonomy assignment.
- Non-English feeds. Slated for a later phase.

7. The data

A frozen extract has been prepared. It is deliberately small enough to iterate on a laptop and deliberately weighted towards the hard cases — the negative pairs are not a random sample of the offer space, they are the pairs that survive blocking, which is the population the model will actually see in production.

Quantity	Value
Offer records (listings)	828
Distinct true SKUs behind them	175
Distinct brands / categories	43 / 35
Labelled training pairs	9,723 (1,254 positive, 12.9%)
Labelled test pairs	4,024 (520 positive, 12.9%)
Offers carrying a usable MPN	35.0%
Offers carrying a UPC	15.8%
Offers with brand field blank	6.0%

7.1 Data dictionary — listings.csv

Column	Type	Description and known issues
listing_id	string	Primary key.
marketplace	string	Source feed. Determines title style; see §2.
title	string	Seller-supplied title. The main signal. Contains marketing padding, inconsistent casing, occasional typos and missing spaces.
brand_raw	string	Seller-declared brand. Blank 6% of the time; otherwise varies in case and may carry a corporate suffix ("Sony Inc.").
category_raw	string	Source marketplace's own taxonomy label. Not harmonised across feeds; blank or generic ("Electronics") in a minority of rows.
price_usd	float	Offer price. Varies within a SKU by seller and condition. Useful as a ratio, dangerous as a level.
condition_raw	string	New / Renewed / Refurbished / Open Box / Used - Like New. Not part of SKU identity — see §5.2.
seller_name	string	Merchant handle. Not identity-bearing.
mpn_raw	string	Manufacturer part number where the seller supplied one. Blank 65% of the time. Punctuation and casing are inconsistent with the same code as it appears in titles. Note that MPN is sometimes a <i>line-level</i> code shared by several SKUs (see the LG television example in §5.5) — an MPN match is strong evidence but not proof.
upc	string	12-digit UPC where supplied. Rare (15.8%) but when both sides have one and they agree, it is close to decisive.
shipping_weight_lb	float	Often blank. Weakly informative.
first_seen_date	date	Ingestion date. Not identity-bearing.

7.2 Data dictionary — pair files

File	Columns	Rows
train_pairs.csv	pair_id, listing_id_a, listing_id_b, label, label_confidence	9,723
test_pairs.csv	pair_id, listing_id_a, listing_id_b	4,024
test_labels.csv	pair_id, label	4,024

label is 1 for same-SKU, 0 otherwise. label_confidence records annotator agreement and is discussed in §8. Column order within a pair is randomised.

7.3 Reference files — analysis only

Do not train on these. reference_listing_sku.csv maps each listing to its true SKU, its product line and its item kind. reference_pair_types.csv tags every pair with the category of hard case it represents. Both exist so that you can slice errors during analysis and report per-class performance. Using either as a feature makes the label trivially recoverable and invalidates every number you produce.

7.4 How the split was made

The train/test split is by **product line**, not by pair. Every offer of every variant of a given product line — and every accessory listed against it — falls on exactly one side of the split. This matters more than it may appear. A random pair-level split would place offers of the same phone in both train and test, letting the model memorise "listings containing the token A2849 are usually duplicates of each other" rather than learning what makes two titles refer to the same thing. Measured on our reference implementation, a random pair-level split inflates average precision by about three and a half points (0.888 against 0.854), and that inflated number does not transfer to newly ingested product lines, which is precisely the case the production system faces every day. Three points sounds small; here it is the difference between clearing the acceptance bar in §9.4 on paper and clearing it in production.

A consequence you should plan for: the test set contains brands, model codes and colour names that never appear in training. Any feature that is a lookup keyed on a specific model string will contribute nothing at test time. Features must generalise over the *shape* of the evidence — "the two model-like tokens conflict" — rather than over specific values.

For the same reason, fit vectorisers, IDF tables and any learned normalisation on the training split only. The corpus statistics of the test listings are not available to you at training time in production.

7.5 Composition of the test set by hard-case category

Pair type	Test rows	Label	What it tests
same_sku	499	1	Positives across title styles, conditions and feeds

token_block_noise	667	0	Easy negatives that share a common token
cross_brand_lookalike	556	0	Same category, different manufacturer
variant_color	511	0	Colour is the only difference
variant_capacity	309	0	Storage or memory is the only difference
variant_multi_attr	311	0	Two or more attributes differ
random_same_category	311	0	Unrelated items from one category
accessory_vs_host	295	0	Accessory against the product it names
variant_size	151	0	Physical size or volume is the only difference
accessory_diff_host	150	0	Accessories for two different hosts
successor_model	143	0	Adjacent generations of one product line
variant_pack	121	0	Pack quantity is the only difference

The training split has the same twelve categories in similar proportion, plus a small `same_brand_diff_line` class.

8. Labelling process and known label noise

Pairs were labelled by the Catalog Operations annotation team against the \$5 specification. Each pair was seen by two annotators; disagreements went to a third. The `label_confidence` column records the outcome:

Value	Meaning	Share of training pairs
high	Both annotators agreed immediately	67.4%
medium	Agreement after one annotator revised	23.7%
low	Went to adjudication; genuine ambiguity	8.9%

Roughly 12% of the low-confidence labels are wrong. An audit of a 100-pair sample put the error rate in that stratum at about one in eight; the high- and medium-confidence strata were clean. This is real and it is not going to be fixed before you start. Practically it means (a) perfect test performance is unattainable and an F1 above roughly 0.85 should make you suspect leakage rather than success, and (b) you have a decision to make about the low-confidence rows: drop them, down-weight them, or train on them as-is. All three are defensible. Whichever you pick, measure the other two — this is the kind of thing that is worth a paragraph in your write-up.

Confidence is supplied for the training split only. The test labels carry no confidence column and must be treated as ground truth for scoring purposes, noise included.

9. Evaluation protocol and success criteria

9.1 Why accuracy and F1 are the wrong headline

The two error types are not symmetric in cost. A false negative leaves two SKU pages where one would do: mild search dilution, a slightly worse customer experience, no incorrect information anywhere. A false positive puts a wrong price, wrong specification or wrong image on a live page, can mis-route an order, and generates a merchant dispute with an SLA attached. Catalog Operations costs a false merge at roughly fifteen times a missed merge.

9.2 Primary metric

Recall at precision ≥ 0.95 , measured on the test split, with the threshold chosen on a validation split carved out of training (grouped by product line, same as \$7.4 — never by random pair). Report the threshold you selected and the validation recall at that threshold alongside the test number, so we can see how much the threshold choice itself is overfitting.

9.3 Secondary metrics

- Average precision (area under the precision–recall curve) — threshold-free view of ranking quality.
- Recall at precision ≥ 0.90 and ≥ 0.98 , for the operating-point discussion.
- Per-pair-type error rate, using `reference_pair_types.csv`. A model that reaches the headline number while still failing badly on `variant_pack` is not shippable, because pack-size errors concentrate in consumables where order volume is highest.
- Symmetry check: score every test pair in both column orders. The maximum absolute difference should be zero. Report it.

9.4 Acceptance criteria for milestone 1

Criterion	Target	Incumbent
Recall at precision ≥ 0.95	≥ 0.55	not reachable at any threshold
Average precision	≥ 0.85	0.383
Per-type error rate, worst class	≤ 0.25	0.60 (variant_size)
Symmetry violation	0	0
Scoring latency, single pair, CPU	≤ 5 ms p99	0.2 ms

An internal reference implementation — about twenty-five hand-built features over a gradient-boosted tree ensemble — reaches average precision 0.854 and recall 0.487 at precision 0.95 on this test split. It is not a ceiling; it was assembled in an afternoon to confirm the targets above are reachable, and its two worst classes (variant_size at 0.245 and successor_model at 0.161) have obvious unexploited structure. Treat it as a sanity check, not a bar.

10. Incumbent baseline (numbers to beat)

Full test-split results for catalog-matcher v2.3 at its production threshold of 0.62:

Metric	Value
True positives / false positives	342 / 513
False negatives / true negatives	178 / 2,991
Precision	0.400
Recall	0.658
F1	0.497
Average precision (ranking by its own score)	0.383
Accuracy	0.828

Error rate by pair type, which is where the diagnosis actually lives:

Pair type	Errors / rows	Error rate	Interpretation
variant_size	91 / 151	0.603	Size tokens are not parsed at all
variant_pack	49 / 121	0.405	Pack phrasings are not normalised
variant_capacity	119 / 309	0.385	Unit conversion missing; MPN rule over-fires
same_sku	162 / 499	0.325	Missed positives: marketing padding depresses Jaccard
variant_color	165 / 511	0.323	Colour synonyms unhandled in both directions
successor_model	26 / 143	0.182	Generation digit invisible to word tokenisation
variant_multi_attr	31 / 311	0.100	Easier — two differences drag Jaccard below threshold
random_same_category	17 / 311	0.055	Handled
token_block_noise	28 / 667	0.042	Handled
accessory_vs_host	2 / 295	0.007	Mostly caught by the brand veto — fragile, see §11
cross_brand_lookalike	1 / 556	0.002	Handled by the brand veto

Two observations worth carrying into the design. First, every category above 0.15 is a case where a specific attribute needs to be *extracted and compared* rather than absorbed into a bag of words. Second, the accessory class looks solved but is not: it is being caught by the brand veto, which works only because most accessory sellers are third-party brands. The same-brand accessories in the catalogue — Dyson filters, Breville filters — slip straight through, and that subclass will grow as first-party accessory feeds are onboarded in Q4.

11. Catalogue of known hard cases

Compiled from the current manual review queue. You are not expected to solve all of these, but your error analysis should be able to say where each one stands.

Capacity and memory

Written as 1TB, 1 TB, 1024GB, 1.0TB, 0.5TB for 512 GB, and 256g for 256 GB. Laptop offers carry compound specifications — 16GB/512GB means RAM and storage, and the two must be compared positionally, not as a set.

Pack quantity

At least eight surface forms in the data: 24 Count, 24 ct, 24ct, Pack of 24, (24-Pack), 24 Pack, 24-Count, 24 pcs. Absence means one, which is different from unknown — consider whether your feature set can express that distinction.

Physical size and volume

Screen sizes appear as 65", 65 Inch, 65-Inch and embedded inside the model code as 0LED65. Cookware and cookers mix imperial and metric in the same category: 6 Qt / 6 Quart / 5.7L; 2 lb / 907g; 5 qt / 4.73L.

Model codes

The same code appears as WH-1000XM5, WH1000XM5, WH 1000XM5 and wh-1000xm5. Word-level tokenisation destroys the distinction between adjacent generations; the difference between a match and a non-match can be the final character of a token. Note also that a shared MPN does not always imply a shared SKU: LG's television MPNs in this extract are line-level codes covering every panel size.

Colour

Marketing names, abbreviations (Blk), word-order flips (Titanium Natural), and outright omission on the terser feeds. Both a false-positive and a false-negative source.

Accessories and compatibility language

Signalled by for, compatible with, fits, and by accessory head nouns — case, cover, sleeve, protector, filter, cable, charger, ear pads, film. Highest token overlap of any negative class. Price ratio is a strong auxiliary signal here, but see \$5.2 before leaning on price.

Title noise

Marketing prefixes and suffixes (NEW SEALED, Free Shipping, US Seller, ★), random capitalisation, character-level typos (Heapdphones), and occasional missing spaces (AppleMacBook). A domain stoplist beyond standard English stopwords is worth building and worth measuring: report overlap features computed with and without it.

12. Engineering constraints

- **Serving environment is CPU-only.** No GPU is provisioned for the matching service and none is planned this year. A fine-tuned transformer cross-encoder is out of scope for milestone 1 for this reason, not because it would not work.
- **Latency budget is 5 ms p99 per pair,** measured end to end including feature extraction. Peak ingestion is roughly 2,000 candidate pairs per second.
- **Decisions must be explainable to Catalog Operations.** When a reviewer disputes a merge, we need to be able to say which evidence drove it. Feature importances and per-feature contributions are sufficient; an opaque embedding distance is not.
- **No dependence on the reference files at inference time.** They do not exist in production.
- **Reproducibility.** Fixed seeds, pinned dependencies, one command to regenerate every number in your report.

13. Deliverables and milestones

#	Deliverable	Notes
1	Feature extraction module	Deterministic, symmetric, unit-tested. One function per feature family, with a docstring stating the hypothesis the feature encodes.
2	Ablation table	Performance as feature families are added in stages, so we can see what each family bought. Start from a single-feature TF-IDF cosine baseline.
3	Trained model plus threshold	With the validation protocol of §9.2 documented.
4	Error analysis	Per-pair-type table, plus a manual read of the fifty highest-loss test pairs grouped into named failure modes.
5	Write-up	Including the low-confidence-label decision from §8 and its measured effect, and a recommendation on whether a cross-encoder should be revisited when GPU capacity exists.

A suggested order of work. Establish the single-feature baseline first and write the number down; everything afterwards is measured as lift over it. Then add feature families in stages — surface overlap, fuzzy string metrics, IDF weighting, extracted attributes, veto flags — and record the table as you go. Resist adding forty features at once: the ablation is more valuable to the team than the final score, because it tells us what to maintain. When the model is decent, sort the validation set by loss and read the worst fifty pairs by hand. They will cluster into three or four recognisable patterns, and each pattern is a missing feature.

Appendix A — File manifest

File	Rows	Purpose
listings.csv	828	Offer records. The feature source.
train_pairs.csv	9,723	Labelled training pairs with annotator confidence.
test_pairs.csv	4,024	Unlabelled test pairs.
test_labels.csv	4,024	Test ground truth. Score against this only at the end.
reference_listing_sku.csv	828	Analysis only — never a feature.
reference_pair_types.csv	13,747	Analysis only — never a feature.

Appendix B — Glossary

Term	Definition
Offer / listing	One merchant's posting of an item on one marketplace.
SKU	A distinct purchasable product as defined in §5. Multiple offers attach to one SKU.
Product line	A family of SKUs sharing a model, differing only in variant attributes.
Blocking	The retrieval stage that proposes candidate pairs, upstream of this model.
Veto attribute	An attribute whose mismatch settles the decision regardless of other evidence.
False merge	A false positive: two different SKUs collapsed into one group.
MPN	Manufacturer part number. Sometimes SKU-level, sometimes line-level.

Meridian Commerce Group — Catalog Intelligence. Internal document CI-2026-014 v1.2. Questions to the Entity Resolution pod. This extract is a frozen September 2026 sample and should not be used for production monitoring.